

VeriFront: 1. Manifest Integrity: How is the manifest itself verified?

Answer: The manifest integrity is verified through a **three-layer verification chain**:

Layer 1 - ENS Root of Trust:

- The ENS domain (e.g., `uniswap.eth`) serves as the root of trust
- ENS contenthash field points to the manifest's IPFS CID
- ENS records are secured by Ethereum blockchain consensus

Layer 2 - Cryptographic Signatures:

- Each manifest is signed using EIP-712 by the ENS domain owner's private key
- The signature covers the entire manifest including all asset hashes
- Clients verify: `signer_address == ENS_owner_address`

Layer 3 - Content Addressing:

- The manifest itself is content-addressed (IPFS CID)
- Any tampering changes the CID, breaking the ENS→Manifest link
- Multiple IPFS nodes can provide consensus on the correct manifest

Verification Flow:

1. User enters: `uniswap.eth`
 2. Resolve ENS → contenthash: `QmManifest123`
 3. Fetch manifest from IPFS (via P2P or gateway)
 4. Verify: `manifest_hash == QmManifest123`
 5. Verify: `manifest.signature` is valid
 6. Verify: `signer == ENS_owner(uniswap.eth)`
 7. Trust established ✓
-

2. Decentralized Retrieval Architecture: How does this improve upon current gateway-based access?

Answer: VeriFront fundamentally changes the trust model from **"trust the gateway"** to **"verify everything"**:

Current Gateway Problems:

- **Trust Required:** Must trust gateway.ipfs.io to serve correct content
- **DNS Vulnerability:** Gateway domain can be hijacked
- **No Verification:** Browser blindly accepts whatever gateway serves
- **Centralization:** Single gateway = single point of failure

VeriFront's Approach:

- **Zero Trust Transport:** Treats ALL transport (P2P, gateways, CDN) as untrusted
- **Mandatory Verification:** Every asset is SHA-256 verified before use
- **Multiple Sources:** Fetches from P2P network first, gateways as fallback
- **No DNS Dependency:** Content identified by hash, not location

Key Difference - Verification Layer:

Current: Browser → DNS → Gateway → [TRUST] → Content

VeriFront: Browser → P2P/Gateway → [VERIFY HASH] → Content

Practical Implementation:

1. **P2P First:** Browser connects directly to other browsers/nodes via WebRTC
 2. **Parallel Fetching:** Simultaneously tries multiple sources
 3. **Hash Verification:** Every asset checked against manifest
 4. **Gateway Fallback:** If P2P fails, uses gateway BUT still verifies
 5. **Result:** Gateway compromise cannot serve malicious content undetected
-

3. Wallet Integration: Specific wallet and why?

Answer:

Primary Target: MetaMask (via Snaps API)

Why MetaMask:

- **Market Share:** 30M+ active users, ~80% wallet market share
- **Snaps API:** Allows custom code execution in isolated environment
- **Technical Fit:** Snaps can intercept dApp loads and perform verification
- **Developer Ecosystem:** Large community familiar with MetaMask

Technical Integration Points:

- **Snap Permissions:** `endowment:network-access`, `endowment:long-running`

- **RPC Methods:** Custom `wallet_verifyFrontend` method
- **UI Integration:** Native MetaMask UI warnings for verification failures

Secondary Targets:

Frame.sh (Desktop):

- **Why:** Developer-focused, supports experimental features
- **API:** Frame Extension API for custom plugins
- **Advantage:** Power users, willing to test new security features

WalletConnect v2 (Mobile):

- **Why:** Protocol-level integration, works with multiple wallets
- **API:** Custom RPC methods over WalletConnect protocol
- **Advantage:** Single integration covers Rainbow, Trust, Argent

Implementation Timeline:

- Month 3: MetaMask Snap prototype
- Month 4: Frame.sh plugin
- Month 5: WalletConnect integration
- Month 6: Production release

4. Performance Targets: Specific technical baseline and goals?

Answer:

Performance Baselines & Targets

Metric	Current Web2 Baseline	VeriFront Target	How We Achieve It
Cold Load (First Visit)	1.2s (CDN)	<3.0s	Parallel P2P + gateway fetch
Warm Load (Return Visit)	0.3s (Browser cache)	<0.5s	Service Worker + IndexedDB cache
Manifest Resolution	N/A (DNS: ~20ms)	<200ms	ENS resolution caching

Asset Verification	0ms (none)	<50ms per MB	Web Crypto API (native speed)
P2P Discovery	N/A	<500ms	DHT + cached peer list
Gateway Fallback Trigger	N/A	<200ms	Instant timeout on P2P
Total Time to Interactive	2.0s	<3.5s	Critical asset prioritization

Technical Optimizations:

1. Parallel Fetching:

- Simultaneously request from 3-5 peers
- Race for first valid response
- No waiting for slow peers

2. Progressive Loading:

- Load critical assets first (HTML, main JS)
- Defer non-critical resources (images, fonts)
- Show UI as soon as critical path verified

3. Intelligent Caching:

- Service Worker: In-memory cache (50MB)
- IndexedDB: Persistent cache (500MB)
- Manifest cache: 24-hour TTL
- Peer cache: Remember fast peers

4. Measurement Method:

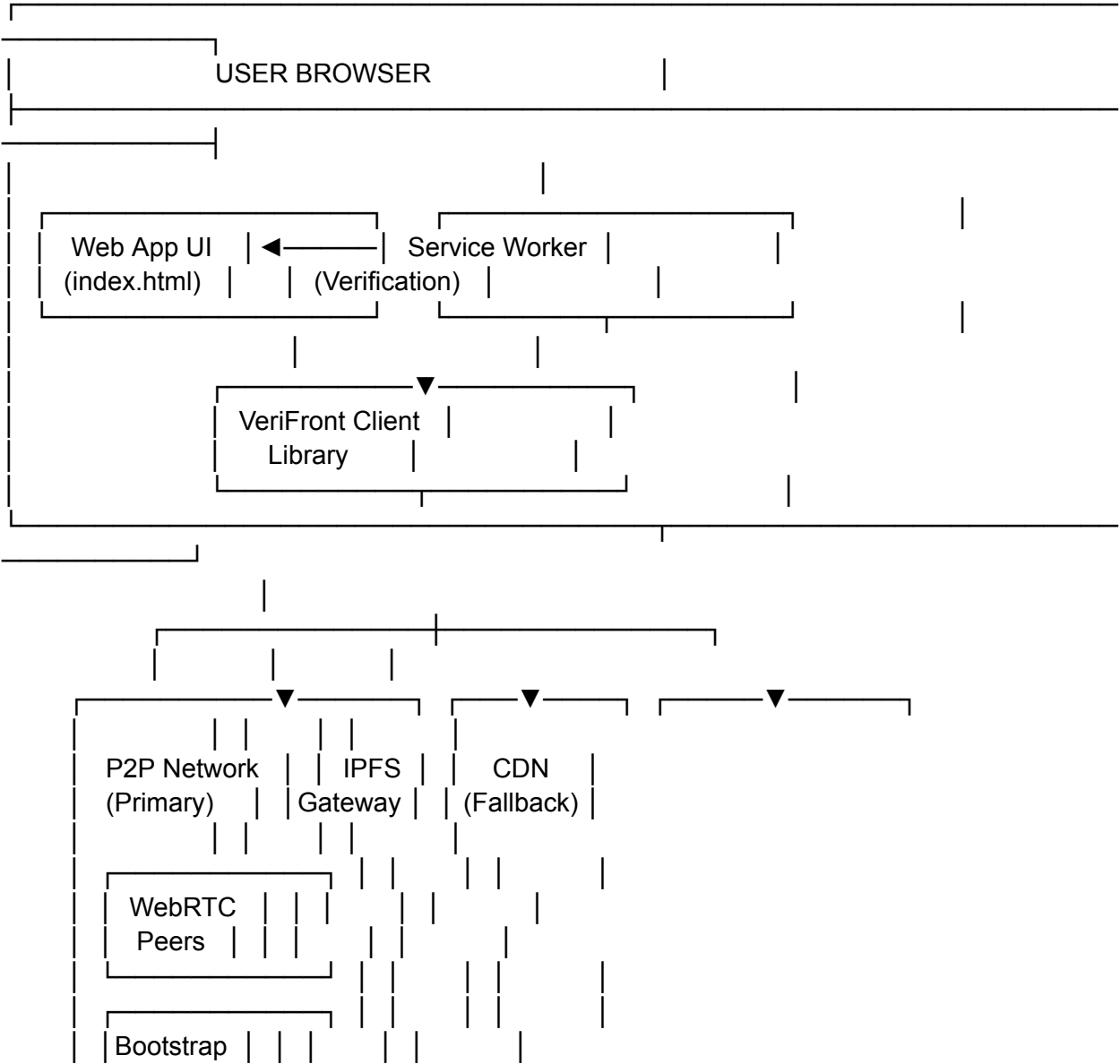
```
// Performance monitoring code
const metrics = {
  manifestFetch: performance.mark('manifest-end') - performance.mark('manifest-start'),
  p2pDiscovery: performance.mark('peers-found') - performance.mark('discovery-start'),
  assetVerification: performance.mark('verify-end') - performance.mark('verify-start'),
  totalLoad: performance.mark('interactive') - performance.mark('navigation-start')
};
```

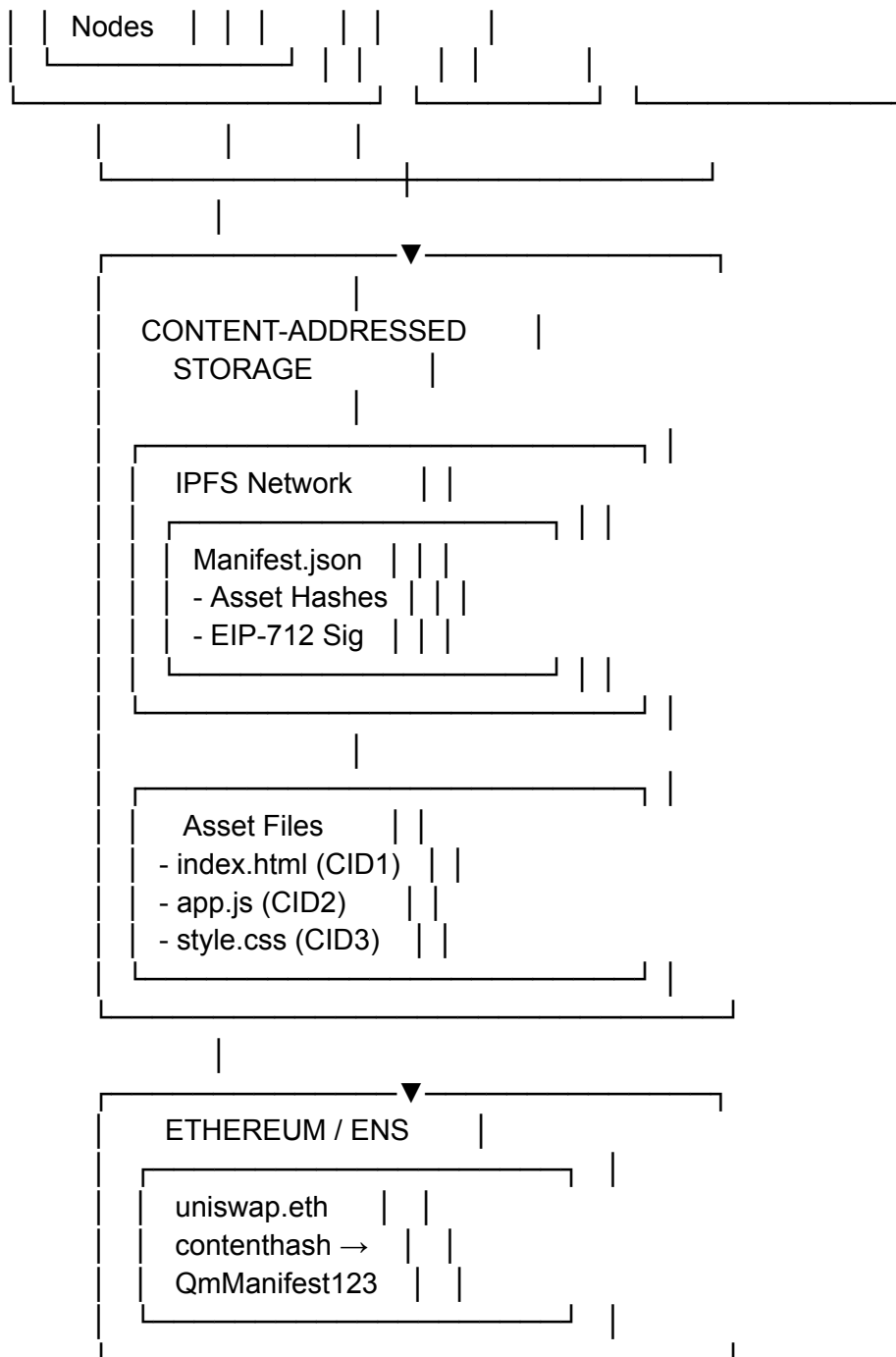
```
// Report to monitoring service
fetch('/metrics', { method: 'POST', body: JSON.stringify(metrics) });
```

Success Criteria:

- P50 (median): <2.5s total load
- P90: <4s total load
- P99: <6s total load
- Verification failure rate: <0.1%
- Successful P2P fetch: >60% of requests

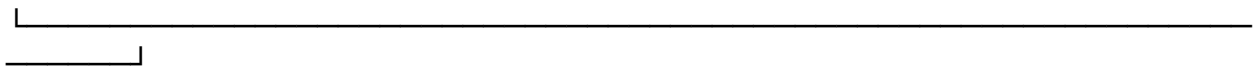
System Architecture Diagram





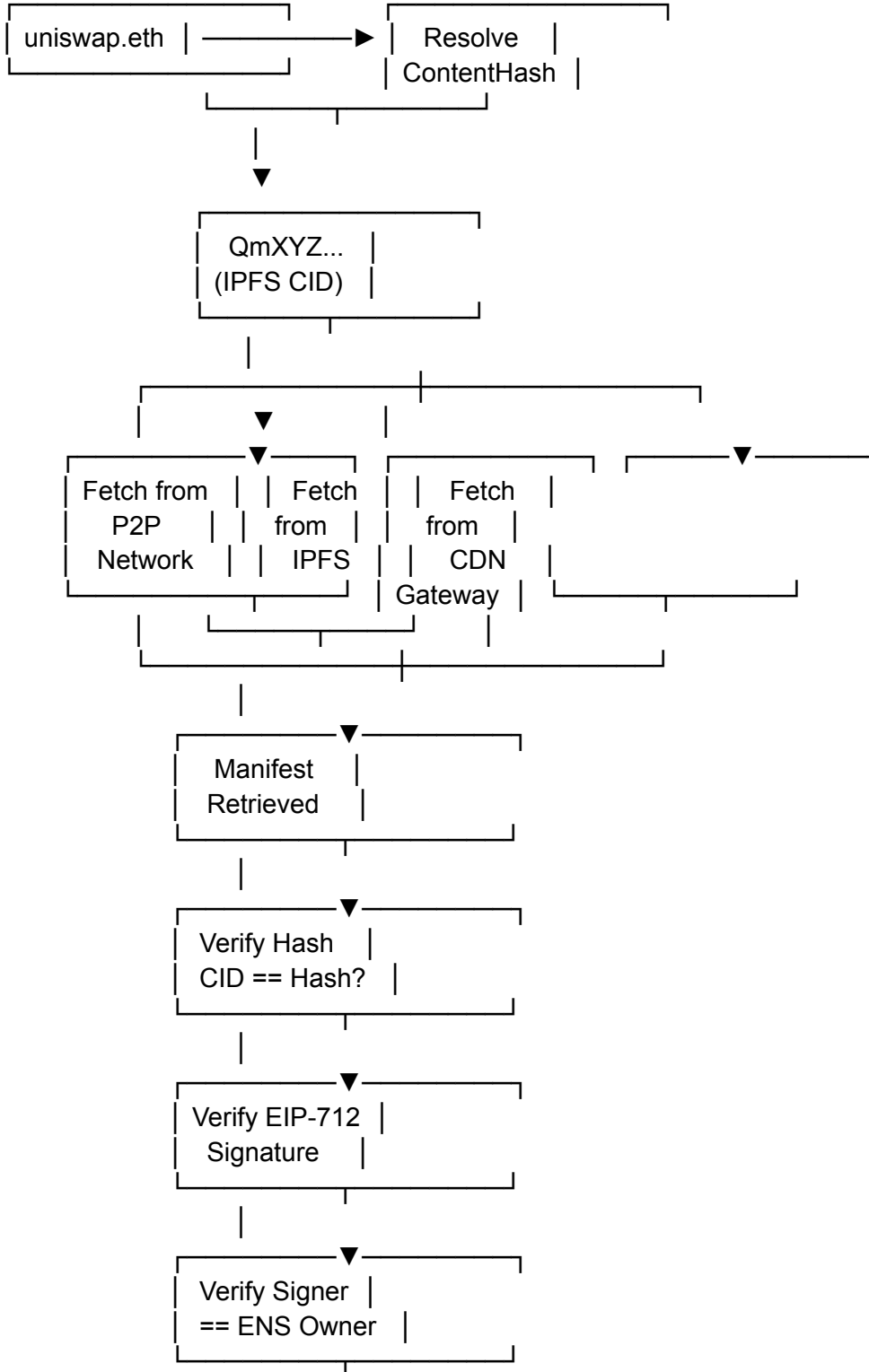
Verification Flow Diagram

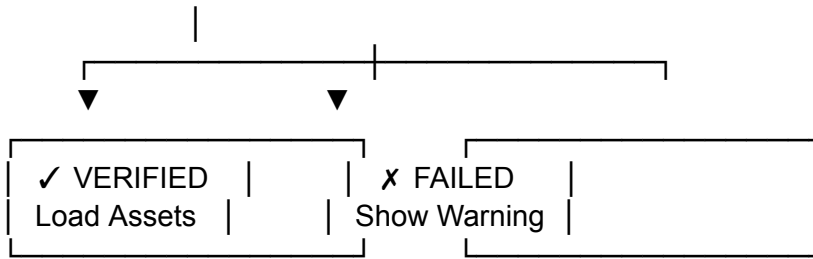




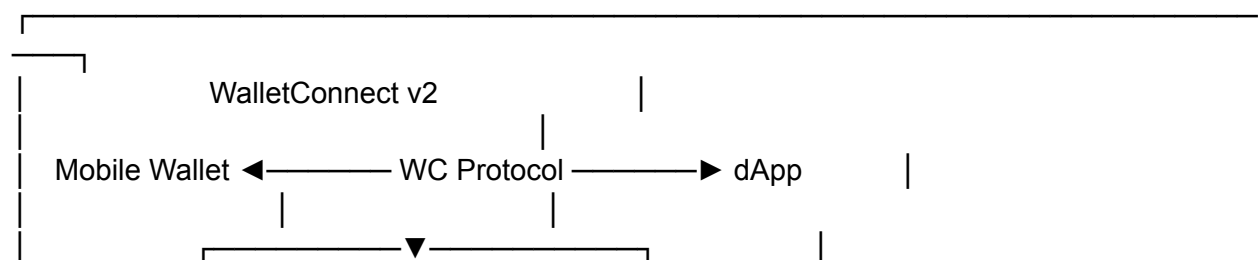
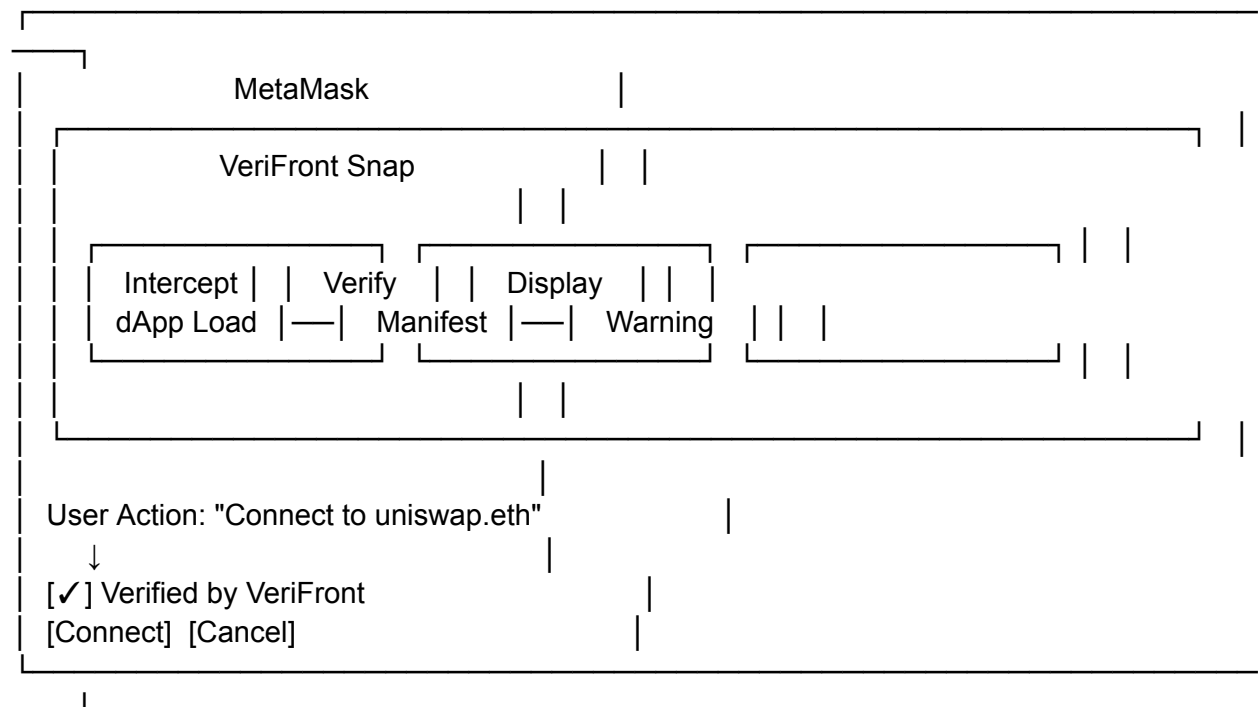
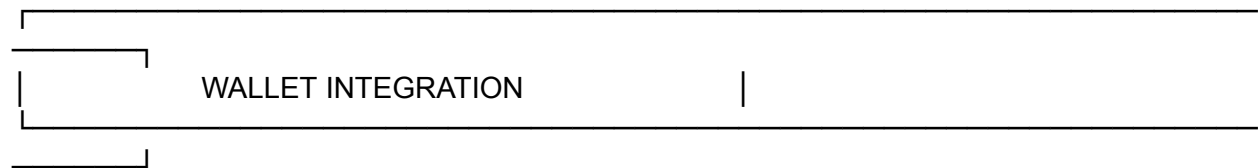
User Input

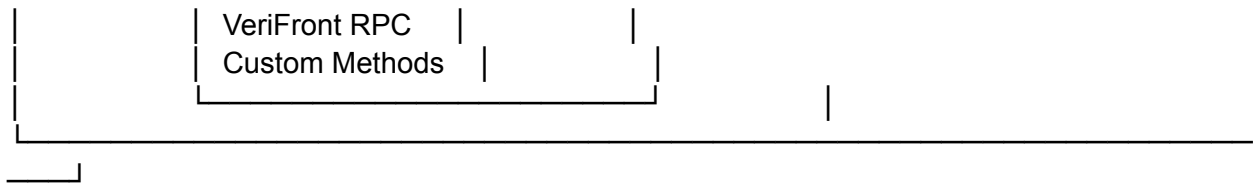
ENS Resolution





Wallet Integration Architecture





Summary

VeriFront solves the critical security vulnerabilities in current "decentralized" applications by implementing a **verification-first architecture** that:

1. **Never trusts the transport layer** - whether P2P, gateway, or CDN
2. **Cryptographically verifies everything** using SHA-256 hashes
3. **Uses ENS + blockchain** as the root of trust
4. **Maintains acceptable performance** through intelligent caching and parallel fetching

The system is designed to be incrementally adoptable, starting with service worker integration (no wallet required) and progressively adding P2P capabilities and wallet verification for maximum security.